



Revisão para a Prova EDII

1. Árvores N-árias

- **Definição Geral:** Uma generalização das árvores binárias onde cada nó pode ter um número arbitrário (ou até n) de filhos. Nos materiais, adota-se a definição generalista de ordem arbitrária e sem ordenação pré-estabelecida para os nós.
- **Representação Parentizada:** Técnica de aninhamento onde cada par de parênteses contém um nó e seus filhos (ex: $(8(15(20()10()28()))23()2(36()7()))$). Se o nó não possui filhos, é seguido por parênteses vazios `()`.
- **Modelagem Primogênito/Irmão:** Uma forma de otimização de memória na qual cada nó possui apenas duas referências:
 1. `filho` : Aponta para o seu primeiro filho (primogênito).
 2. `irmao` : Aponta para o irmão imediatamente ao lado.
 - *Vantagem:* Os filhos de um nó passam a ser representados por uma lista simplesmente encadeada, evitando o desperdício de vetores de tamanho fixo.
- **Modelagem via Lista de Filhos:** Abordagem alternativa comum em linguagens orientadas a objetos, onde cada nó armazena explicitamente uma coleção dinâmica (como um `ArrayList`) contendo referências para todos os seus nós filhos.

2. Tries - Árvores de Prefixos

- **Conceito:** Uma Trie (do inglês *re-trie-val*) é uma árvore n -ária especializada e altamente eficiente projetada para armazenar e recuperar chaves do tipo string.
- **Estrutura de Nós:** * A raiz não armazena caracteres, servindo puramente como ponto de partida.

- Os caracteres não são guardados diretamente nos nós, mas são **inferidos através do índice das arestas** (ponteiros do vetor de filhos).
- Cada nó contém um vetor de referências para os filhos (de tamanho igual ao alfabeto adotado, ex: 26 para letras minúsculas) e uma flag booleana (`isChave`) para indicar se o caminho percorrido da raiz até ali constitui uma palavra válida.
- **Desempenho:** Operações de busca, inclusão e remoção possuem complexidade de tempo de $\mathcal{O}(m)$, onde m é o comprimento da palavra. O custo depende unicamente do tamanho da string manipulada, independentemente de haver 10 ou 10 milhões de chaves armazenadas na árvore.
- **Aplicações Comuns:** Mecanismos de autocompletar, corretores ortográficos, roteamento de redes de computadores e algoritmos de compressão.
- **Algoritmo de Exclusão (Bottom-Up):** A remoção é implementada de forma recursiva e possui três cenários fundamentais:
 1. *Chave única sem prefixos compartilhados:* Remove-se fisicamente os nós de trás para frente até a raiz.
 2. *Chave é prefixo de uma palavra maior:* Apenas desmarca-se a flag `isChave` para `false`, sem desalocar nenhum nó da memória.
 3. *Chave compartilha prefixo, mas possui um sufixo exclusivo:* Apagam-se os nós correspondentes ao sufixo exclusivo até alcançar o nó que pertence à outra palavra.

3. Grafos - Conceitos e Topologia

- **Definição Formal:** Um grafo é uma estrutura matemática representada pelo par ordenado $G = (V, A)$, composto por um conjunto finito de vértices V e um conjunto de arestas ou arcos A .
- **Tipos de Grafos:**
 - **Grafo Não Direcionado (GND):** As ligações são arestas bidirecionais simétricas. Se existe a aresta $\{u, v\}$, significa que u está ligado a v e vice-versa.

- **Grafo Direcionado (GD / Dígrafo):** As ligações são arcos com sentido definido. Cada arco é um par ordenado (u, v) , indicando uma direção de saída de u com destino a v .
- **Grafo Simples:** Não possui laços (arestas que conectam um vértice a ele mesmo) nem arestas paralelas (múltiplas arestas ligando o mesmo par de vértices).
- **Grafo Completo (K_n):** Grafo simples no qual existe exatamente uma aresta ligando cada par de vértices distintos.
- **Grafo Regular:** Grafo onde todos os vértices possuem rigorosamente o mesmo grau. *Nota:* Todo grafo completo é regular.
- **Grafo Bipartido:** Quando o conjunto de vértices V pode ser dividido em dois subconjuntos disjuntos V_1 e V_2 , de tal forma que todas as arestas do grafo conectem apenas um vértice de V_1 a um vértice de V_2 . Ele será **Bipartido Completo ($K_{|V_1|, |V_2|}$)** se cada vértice de V_1 se conectar a absolutamente todos os vértices de V_2 .
- **Grafo Complemento ($\overline{G}$):** Um grafo formado com os mesmos vértices de G , mas cujas arestas são exatamente aquelas que faltam em G para que ele se torne um grafo completo.
- **Grau de um Vértice:**
 - Em GND, o grau $d(i)$ é a quantidade de arestas incidentes no vértice i .
 - Em GD, divide-se em grau de entrada $d^-(i)$ (arestas que chegam) e grau de saída $d^+(i)$ (arestas que partem).
 - **Teorema do Aperto de Mãos (Handshaking Theorem):** A soma dos graus de todos os vértices de um GND é igual ao dobro do seu número de arestas ($\sum d(i) = 2m$). *Corolário:* O número de vértices com grau ímpar em qualquer GND é obrigatoriamente par.
- **Conectividade e Passeios:**
 - **Passeio:** Uma sequência arbitrária alternada de vértices e arestas conectadas. Pode ser *aberto* (início e fim distintos) ou *fechado* (início e fim iguais).
 - **Cadeia:** Um passeio que proíbe a repetição de arestas.

- **Caminho:** Uma cadeia que proíbe a repetição de vértices (com exceção do primeiro e do último no caso de caminhos fechados).
- **Ciclo:** Um caminho fechado (o vértice inicial coincide com o final).
- **Grafo Conexo:** Quando existe pelo menos um caminho válido ligando cada par de vértices (u, v) do grafo. Se for desconexo, ele é composto por duas ou mais componentes conexas isoladas.
- **Métricas de Ciclos:**
 - **Cintura:** O comprimento (número de arestas) do menor ciclo presente no grafo.
 - **Circunferência:** O comprimento do maior ciclo presente no grafo.
 - **Corda:** Uma aresta que interliga dois vértices não consecutivos pertencentes a um ciclo.
- **Isomorfismo de Grafos:** Dois grafos G e H são isomorfos se houver uma bijeção (correspondência um-para-um) entre seus vértices e arestas preservando integralmente as relações de adjacência.
 - *Condições necessárias (mas não suficientes):* Devem possuir a mesma quantidade de vértices, de arestas, de componentes conexas e a mesma distribuição de graus dos vértices.

4. Representação Computacional de Grafos

- **Matriz de Adjacências:** Matriz quadrada $A_{n \times n}$ onde $a_{ij} = 1$ se existe aresta entre os vértices i e j , e 0 caso contrário.
 - *Vantagem:* Verificação de adjacência extremamente rápida em tempo constante $\mathcal{O}(1)$.
 - *Desvantagem:* Desperdício severo de memória ocupando espaço $\Theta(n^2)$, mesmo para grafos esparsos (poucas arestas). É simétrica para GNDs.
- **Lista de Adjacências:** Um vetor ou estrutura indexada de tamanho n (número de vértices) onde cada posição aponta para uma lista encadeada dinâmica contendo apenas os vizinhos diretos daquele nó.
 - *Vantagem:* Uso eficiente de memória ocupando espaço proporcional ao tamanho real do grafo $\mathcal{O}(n + m)$.

- *Desvantagem:* A busca para determinar se dois nós específicos são adjacentes pode exigir varrer a lista inteira, sendo limitada por $\mathcal{O}(n)$.

Questões de Fixação

1. **(Grafos/Handshaking)** Um escultor deseja criar um monumento colocando 7 pilares em círculo e ligando cada pilar a exatamente outros 3 pilares por fios de ouro. Explique por que essa estrutura é matematicamente impossível. (Dica: Pense no Corolário do Teorema do Aperto de Mãos).
2. **(Árvores N-árias)** Desenhe a árvore n-ária representada pela seguinte expressão parentizada:
 $A(B(E()F()G())C(H()I()J())D(K()L()M(N(O()P()))))$. Em seguida, simule visualmente como ficariam os ponteiros desta mesma árvore utilizando a modelagem **Primogênito/Irmão**.
3. **(Tries)** Suponha que uma Trie contenha as strings "MAR" e "MARCA". Descreva detalhadamente o comportamento das flags e dos nós caso a função `excluir("MAR")` seja executada.
4. **(Matrizes)** Desenhe a **Matriz de Adjacências** e a **Lista de Adjacências** para um grafo ciclo simples C_4 .